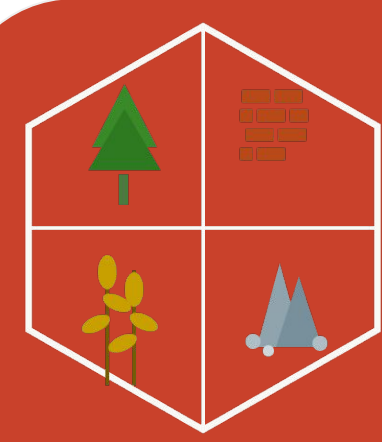




Figure 1:
Catan Board



CaRL - Catan RL

Arrush Shah and Suraj Kidiyoor



Introduction

Catan is a strategy board game (Figure 1) where players collect resources, build roads and settlements, and compete to earn victory points. It is difficult for traditional AI because it requires resource management, expansion, trading, and long-term planning with random dice rolls. Our project uses reinforcement learning to train an agent that improves through repeated games instead of relying only on

Novelty

Unlike simpler board games, Catan includes hidden information, random dice rolls, multi-step planning, and changing player interactions. Our approach adapts reinforcement learning to this high-dimensional, partially stochastic environment. We designed a custom reward function that balances short-term progress, such as building settlements and cities, with long-term goals like gaining victory points. We also used action masking so the model only selects legal moves, making training more efficient and realistic.

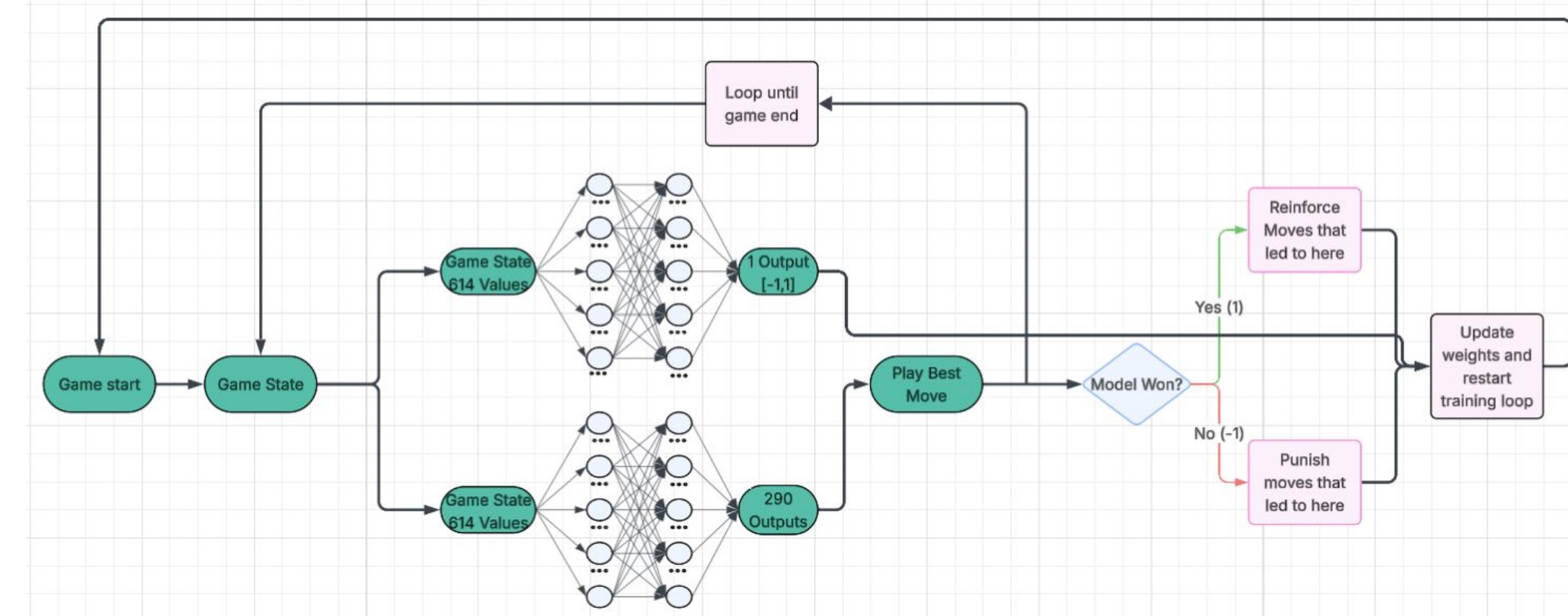


Figure 2: Training Architecture

Methods

The training process follows Figure 2. Catanatron, our environment backend, first converts the current Catan board into a numerical game state that includes player resources, victory points, roads, settlements, cities, board layout, and available actions. This encoded state becomes the input to our Maskable PPO reinforcement learning model, allowing the agent to make decisions from the same type of information a player would use during a game.

The model contains two main outputs: a policy network and a value network. The policy network scores possible actions, while the value network estimates how strong the current board position is. Because Catan has a large action space where most actions are illegal on any given turn, we used action masking to remove invalid moves before the model selects an action. This prevents the agent from wasting training time on impossible moves and keeps learning focused on realistic gameplay decisions.

After the agent chooses a legal move, Catanatron updates the board and returns a reward. We tested multiple reward functions while keeping the same architecture from Figure 2. Instead of only rewarding a final win, our final reward function gave feedback for progress such as gaining victory points, building settlements, upgrading cities, and winning. The model repeatedly used this feedback to update its weights over many simulated games.



Figure 3: Website to Play Against our Model

Results

Our testing shows that reward design strongly affected performance. In one-hour tests against random bots, the basic reward won 25%, the long-term reward dropped to 7%, the third reward reached 36%, and the improved reward reached 66%. With more training, the model stayed strong against random and weighted-random bots and reached about 80% against MCTS, but only 5% against ValueFunction (Figure 4). This shows the model improved past simple bots but still struggled against stronger strategy-based opponents. The need for more training and tuning is necessary for the advancement of CaRL.

Impact & Future Work

This project shows how reinforcement learning can be used in Catan, a game with dice randomness, long-term planning, and changing legal moves. The website in Figure 3 turns the trained model into a playable demo, making the project easier to understand.

Future work includes improving trading, settlement placement, resource rewards, and testing against stronger Catanatron bots.

Figure 4:
Model Testing as it Trains

